

Databasesystems project

By Anonyms

DM576 - Databasesystems

Teacher: Panagiotis Tampakis

Contents

1	Introduction	3
2	Database specifications	3
3	Relational Schema	3
4	Part 1: Database Design	4
4.1	Conceptual Design:	4
4.2	Normalization	5
5	Part 2: Database Design	6
5.1	Queries	6
5.2	Defining the database	12
5.2.1	Streaming_Platform	12
5.2.2	Users	12
5.2.3	Free_User	12
5.2.4	Premium_User	12
5.2.5	Creator_User	12
5.2.6	Content	13
5.2.7	Movie	13
5.2.8	TV_Show	13
5.2.9	Season	13
5.2.10	Episode	13
5.2.11	Watch_History	13
5.2.12	Trigger enforce_tot_duration_limit	14
6	Part 3: Querying	15
6.0.1	i - Movies and TV-Shows	15
6.0.2	ii - Counting uploads	17
6.0.3	iii - Complete views	18
6.0.4	iv - Max views on date	19
6.0.5	v - UserRecord	20
7	Part 4 - Triggers	22
7.1	Updating UserRecord	22
7.2	Insertion	22
7.3	Deletion	24
7.4	Updating Creator_User(total_views)	26
7.5	Updating Creator_User(uploaded_content_count)	27
7.6	Updating Content	28
8	Part 5 - Index	29
8.1	Defining indexing	29
8.1.1	ii - Counting uploads	30
8.1.2	iii - Complete views	31
8.1.3	iv - Max views on date	32

1 Introduction

This report includes all 5 assignments in the Database Systems course project.

2 Database specifications

Let's assume a DB of a content streaming meta-service of platforms like Netflix, Disney+, etc. This database is able to store content from both various platforms and individual users. Users of this meta-service can watch and/or upload new content. A content can appear in at most one platform. Content can be either a Movie or a TV Show Episode. However, the database should be expandable to other categories also. A TV Show has at least one season and a season has at least one episode. A User can be either a Free User, a Premium User or a Creator User. A Creator User can both watch and upload content. A user can either watch the same content multiple times by restarting or to continue where she/he left off. Attribute is Completed is a boolean that becomes TRUE if the user has reached the end of the content.

3 Relational Schema

- Streaming_Platform(platform_id, name, headquarters, monthly_active_users, revenue)
- Content(content_id, platform_id, type, duration, IsFree, uploadedBy)
- Movie(content_id, title, release_year, genre, director, rating)
- Episode(content_id, show_id, season_number, episode_number, title, release_date)
- Season(season_number, show_id, release_date)
- TV_Show(show_id, title, release_year, genre, director)
- Users(user_id, first_name, last_name, email, birthdate, country, type)
- Free_User(user_id, ad_frequency, max_resolution)
- Premium_User(user_id, subscription_plan, monthly_fee, payment_method, renewal_date)
- Creator_User(user_id, uploaded_content_count, total_views, earnings)
- Watch_History(user_id, content_id, session_start_timestamp, from_t, to_t, IsCompleted)

4 Part 1: Database Design

4.1 Conceptual Design:

Convert the relational schema into conceptual-level design (E-R diagram), i.e. following a process inverse to classic DB design. Argue about the decisions you made to consider something as an entity, relationship, attribute etc. You should include in the report the E-R Diagram accompanied with a description and any assumptions that you made

Answer:

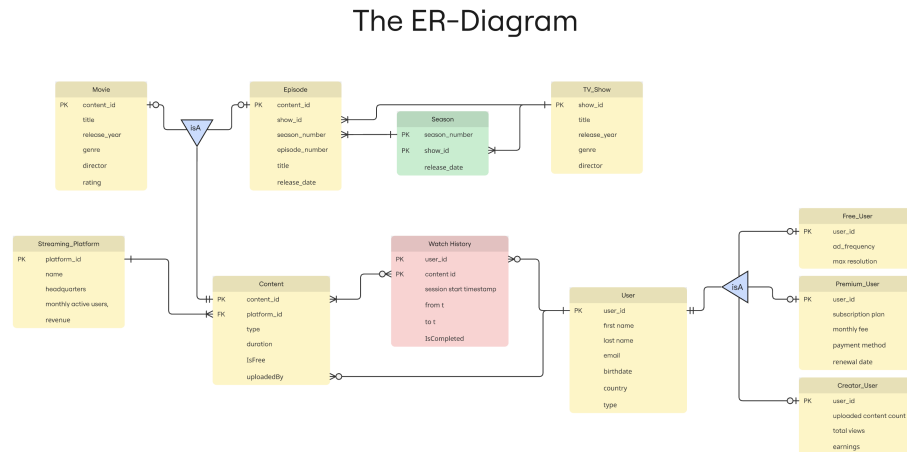


Figure 1: ER-Diagram for Streaming_platform

The ER-Diagram above depicts the relations of a streaming platform. Each table/relation from the database is considered an entity, that has some attributes, that tells something about the specific entity. The entity, Streaming_platform has a one to many relationships to the entity Content, meaning that a piece of content can only exist on one streaming platform at a time. Also, one streaming platform can have many pieces of content. The Content entity has a one and only one to one or zero relationships to both Movie and Episode. These pass through an isA, which indicates that a piece of content must be either be a Movie or an Episode, not both. While a movie is just a movie, the episode entity has a many to one relationship to Season, as well as a many to one relationship to TV_Show. This means that one or more episodes can be point to a season and a tv-show. The Season entity also has a many to one relationship to TV_Shows, as on tv-show can hold one or more seasons. Note that this makes the Season entity a weak entity since you would need both the season_number and the show_id to identify a season

The Content entity also points to User, either through Watch_History or directly to a user, if a user has uploaded that piece of content. The Content entity has a one or many to zero or many to the Watch_History entity. This means that a piece of content have have multiple watch histories at one time (by different users). The Watch_History entity has a zero or many to one relationship to the User entity. This means that a watch history can only exist to one user at a time, but a user can have multiple watch histories to different contents. Note this makes Watch_History a connecting entity as well as a weak entity since it can only be identified by both the user_id and the content_id. The Content entity also has a zero or many to one relationship to the User entity, since a user (Creator_User) can upload zero or more pieces of content

The User entity has a one and only one to one or zero relationship to Free_User, Premium_User and Creator_User. These pass through an isA, which indicates that on user can either be a Free_User, Premium_User and Creator_User. The user can only be one type at a time.

4.2 Normalization

Let us assume one relation that holds all the information about a user's record: UserRecord(user_id, first_name, last_name, content_id, IsUploaded, title, show_id, season_number, episode_number, number_of_views). Attribute IsUploaded is a boolean that becomes true if the specific user has uploaded the content and number_of_views is a number indicating how many times a user has viewed the specific content (if some user has watched a percentage of some content, e.g. half of it, then this should be depicted in this number, e.g. 0.5). Identify the functional dependencies and based on them find if this relation is in BCNF. If it is not, decompose it into BCNF. Argue if the final decomposition is also in 3NF. If it is not, what do we need to change to get it into 3NF? You should document in the report any steps that you took to get to the result.

Answer:

User_Record(user_id, first_name, last_name, content_id, IsUploaded, title, show_id, season_number, episode_number, number_of_views) has the following FS's

By naming all attributes $A, B, C \dots$ we get a better overview.

- $A = \text{user_id}$
- $B = \text{first_name}$
- $C = \text{last_name}$
- $D = \text{content_id}$
- $E = \text{IsUploaded}$
- $F = \text{title}$
- $G = \text{show_id}$
- $H = \text{season_number}$
- $I = \text{episode_number}$
- $J = \text{number_of_views}$

Firstly, check the closure of the FD's. For this:

$A \rightarrow BC, D \rightarrow FGHI, AD \rightarrow ABCDEFGHIJ$.

AD is a superkey. but the other two **violate** BCNF, so these should be decomposed. when decomposing we use the following $R_1 = X^+$ and $R_2 = R - (X^+ - X)$.

- $R_1 = \{A\}^+ \rightarrow \{ABC\}$

This relation contains the dependency $A \rightarrow BC$

A is now a key in R_1 , so it satisfies BCNF.

- $R_2 = R - (\{A\}^+ - A) = \{ADEF GHIJ\}$

This relation contains the dependency $D \rightarrow DFGHI$, this violates BCNF.

This should be decomposed (R_3 and R_4)

- $R_3 = \{D\}^+ = \{DFGHI\}$

This relation contains the dependencies $D \rightarrow DFGHI$, this violates BCNF.

This should be decomposed (R_5 and R_6)

- $R_4 = R_2 - (D^+ - D) = R_4(ADEJ)$

This relation contains the dependencies $A \rightarrow ABC$ and $D \rightarrow DFGHI$, which makes a key this satisfies BCNF.

The resulting relations are: $R_1 = \{ABC\}, R_3 = \{DFGHI\}, R_4 = \{ADEJ\}$. BCNF is stricter than 3NF. Since BCNF has a superkey on the left hand side this also hold for 3NF

5 Part 2: Database Design

Database Implementation: Deploy the DB in PostgreSQL with appropriate commands to create the tables, by considering and enforcing all the integrity constraints and any other constraints you might find useful. The latter can be enforced with the use of checks or triggers. Enter realistic (5-10) records in each of the tables. You should document in the report the SQL queries that you used to CREATE the database and also include any assumptions and justification for your choices. You should also include screenshots of the database tables containing data.

Answer:

5.1 Queries

```
1 CREATE TABLE Streaming_Platform (  
2     platform_id          INT,  
3     name                 VARCHAR(20),  
4     headquarters         VARCHAR(120),  
5     monthly_active_users INT,  
6     revenue              DECIMAL(15,2)  
7 );  
8  
9 ALTER TABLE Streaming_Platform  
10     ADD PRIMARY KEY (platform_id),  
11     ALTER COLUMN name SET NOT NULL,  
12     ALTER COLUMN headquarters SET NOT NULL,  
13     ALTER COLUMN monthly_active_users SET NOT NULL,  
14     ALTER COLUMN revenue SET NOT NULL,  
15     ADD CONSTRAINT chk_mon_act_users CHECK (monthly_active_users >= 0),  
16     ADD CONSTRAINT chk_revenue CHECK (revenue >= 0);  
17  
18 INSERT INTO Streaming_Platform VALUES  
19     (1, 'StreamX', 'New York, USA', 1200000, 85.60),  
20     (2, 'CineWave', 'London, UK', 950000, 70.20),  
21     (3, 'FlixZone', 'Toronto, Canada', 800000, 55.00),  
22     (4, 'MegaStream', 'Tokyo, Japan', 500000, 60.30),  
23     (5, 'VibePlay', 'Berlin, Germany', 650000, 48.70);  
24  
25 CREATE TABLE Users (  
26     user_id          INT,  
27     first_name        VARCHAR(50),  
28     last_name         VARCHAR(20),  
29     email             VARCHAR(50),  
30     birthdate         DATE,  
31     country           VARCHAR(50),  
32     type              VARCHAR(20)  
33 );  
34  
35  
36  
37  
38  
39  
40  
41  
42
```

```

43 ALTER TABLE Users
44     ADD PRIMARY KEY (user_id),
45     ALTER COLUMN first_name SET NOT NULL,
46     ALTER COLUMN last_name SET NOT NULL,
47     ALTER COLUMN email SET NOT NULL,
48     ALTER COLUMN birthdate SET NOT NULL,
49     ALTER COLUMN country SET NOT NULL,
50     ALTER COLUMN type SET NOT NULL,
51     ADD CONSTRAINT unq_email UNIQUE (email),
52     ADD CONSTRAINT chk_type CHECK (type IN ('Free', 'Premium', 'Creator'))
53
54 INSERT INTO Users VALUES
55     (1, 'Alice', 'Smith', 'alice@example.com', '1990-05-12', 'USA', 'Free'),
56     (2, 'Anne', 'Hoej', 'ahoe@imada.sdu.dk', '1963-02-13', 'Denmark', 'Free'),
57     (3, 'Charlie', 'Lee', 'charlie@example.com', '1992-08-21', 'Canada',
58     → 'Premium'),
59     (4, 'Diana', 'Ray', 'diana@example.com', '2000-03-15', 'Japan', 'Premium'),
60     (5, 'George', 'Lucas', 'George@lfilms.com', '1944-05-14', 'USA', 'Creator');
61
62 CREATE TABLE Free_User (
63     user_id          INT,
64     ad_frequency     FLOAT,
65     max_resolution   INT
66 );
67
68 ALTER TABLE Free_User
69     ADD PRIMARY KEY (user_id),
70     ALTER COLUMN ad_frequency SET NOT NULL,
71     ALTER COLUMN max_resolution SET NOT NULL,
72     ADD FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE ON UPDATE
73     → CASCADE,
74     ADD CONSTRAINT chk_ad_freq CHECK (ad_frequency >= 0),
75     ADD CONSTRAINT chk_resolution CHECK (max_resolution <= 720);
76
77 INSERT INTO Free_User VALUES
78     (1, 5.0, 480),
79     (2, 3.0, 720);
80
81 CREATE TABLE Premium_User (
82     user_id          INT,
83     subscription_plan VARCHAR(20),
84     monthly_fee      DECIMAL(5,2),
85     payment_method    VARCHAR(20),
86     renewal_date      DATE
87 );
88
89 ALTER TABLE Premium_User
90     ADD PRIMARY KEY (user_id),
91     ALTER COLUMN subscription_plan SET NOT NULL,
92     ALTER COLUMN monthly_fee SET NOT NULL,
93     ALTER COLUMN payment_method SET NOT NULL,
94     ALTER COLUMN renewal_date SET NOT NULL,
95     ADD FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE ON UPDATE
96     → CASCADE,
97     ADD CONSTRAINT chk_monthly_fee CHECK (monthly_fee >= 0);

```

```

96 INSERT INTO Premium_User VALUES
97     (3, 'Gold', 14.99, 'Credit Card', '2025-06-01'),
98     (4, 'Standard', 9.99, 'PayPal', '2025-06-15');
99
100 CREATE TABLE Creator_User (
101     user_id            INT,
102     uploaded_content_count INT,
103     total_views        INT,
104     earnings           DECIMAL(15,2)
105 );
106
107 ALTER TABLE Creator_User
108     ADD PRIMARY KEY (user_id),
109     ALTER COLUMN uploaded_content_count SET NOT NULL,
110     ALTER COLUMN total_views SET NOT NULL,
111     ALTER COLUMN earnings SET NOT NULL,
112     ADD FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE ON UPDATE
113     → CASCADE;
114
114 INSERT INTO Creator_User VALUES
115     (0005, 5, 250000000, 120000000.50);
116
117 CREATE TABLE Content (
118     content_id          INT,
119     platform_id         INT,
120     type                VARCHAR(20),
121     duration            DECIMAL(5,2),
122     IsFree              BOOLEAN,
123     uploadedBy          INT
124 );
125
126 ALTER TABLE Content
127     ADD PRIMARY KEY (content_id),
128     ALTER COLUMN type SET NOT NULL,
129     ALTER COLUMN duration SET NOT NULL,
130     ALTER COLUMN IsFree SET NOT NULL,
131     ALTER COLUMN uploadedBy SET NOT NULL,
132     ADD FOREIGN KEY (platform_id) REFERENCES Streaming_Platform(platform_id)
133     ON DELETE CASCADE ON UPDATE CASCADE,
134     ADD FOREIGN KEY (uploadedBy) REFERENCES Creator_User(user_id)
135     ON DELETE CASCADE ON UPDATE CASCADE,
136     ADD CONSTRAINT chk_content_type CHECK (type IN ('Movie', 'Episode'));
137
138 INSERT INTO Content VALUES
139     (1, 1, 'Movie', 130.0, FALSE, 5),
140     (2, 2, 'Movie', 130.0, TRUE, 5),
141     (3, 1, 'Movie', 130.0, FALSE, 5),
142     (4, 3, 'Movie', 130.0, TRUE, 5),
143     (5, 4, 'Movie', 130.0, FALSE, 5),
144     (6, 1, 'Episode', 40.0, FALSE, 5),
145     (7, 2, 'Episode', 40.0, TRUE, 5),
146     (8, 1, 'Episode', 40.0, FALSE, 5),
147     (9, 3, 'Episode', 40.0, TRUE, 5),
148     (10, 4, 'Episode', 40.0, FALSE, 5);
149
150

```



```

151 CREATE TABLE Movie (
152     content_id          INT,
153     title                VARCHAR(50),
154     release_year        INT,
155     genre                VARCHAR(20),
156     director             VARCHAR(50),
157     rating               DECIMAL(3,1)
158 );
159
160 ALTER TABLE Movie
161     ADD PRIMARY KEY (content_id),
162     ALTER COLUMN title SET NOT NULL,
163     ALTER COLUMN release_year SET NOT NULL,
164     ALTER COLUMN director SET NOT NULL,
165     ADD FOREIGN KEY (content_id) REFERENCES Content(content_id)
166         ON DELETE CASCADE ON UPDATE CASCADE,
167     ADD CONSTRAINT chk_movie_rating CHECK (rating >= 0.0 AND rating <= 10.0);
168
169 INSERT INTO Movie VALUES
170     (1, 'Star Wars: A New Hope', 1977, 'Sci-Fi', 'George Lucas', 8.2),
171     (2, 'Star Wars: Empire Strikes Back', 1980, 'Sci-Fi', 'Irvin Kershner', 7.5),
172     (3, 'Star Wars: Revenge of the Jedi', 1983, 'Sci-Fi', 'Richard Marquand', 8.0),
173     (4, 'Star Wars: Attack of the Clones', 2002, 'Sci-Fi', 'George Lucas', 7.9),
174     (5, 'Star Wars: Revenge of the Sith', 2005, 'Sci-Fi', 'George Lucas', 9.3);
175
176 CREATE TABLE TV_Show (
177     show_id              INT,
178     title                VARCHAR(50),
179     genre                VARCHAR(20),
180     director             VARCHAR(50)
181 );
182
183 ALTER TABLE TV_Show
184     ADD PRIMARY KEY (show_id);
185
186 INSERT INTO TV_Show VALUES
187     (1, 'Mystery Manor', 'Mystery', 'Jane Doe'),
188     (2, 'Space Chronicles', 'Sci-Fi', 'John Smith'),
189     (3, 'Comedy Central', 'Comedy', 'Lisa Ray'),
190     (4, 'Historical Nights', 'History', 'Alan Turing'),
191     (5, 'Fantasy World', 'Fantasy', 'Diana Prince');
192
193 CREATE TABLE Season (
194     season_number        INT,
195     show_id              INT,
196     release_date         DATE
197 );
198
199 ALTER TABLE Season
200     ADD PRIMARY KEY (season_number, show_id),
201     ADD FOREIGN KEY (show_id) REFERENCES TV_Show(show_id) ON DELETE CASCADE ON
202     ↪ UPDATE CASCADE;
203
204
205

```

```

206 INSERT INTO Season VALUES
207     (1, 1, '2022-01-15'),
208     (1, 2, '2021-07-10'),
209     (1, 3, '2020-03-05'),
210     (1, 4, '2023-02-14'),
211     (1, 5, '2021-11-11');
212
213 CREATE TABLE Episode (
214     content_id          INT,
215     show_id             INT,
216     season_number       INT,
217     episode_number      INT,
218     title               VARCHAR(50),
219     release_date        DATE
220 );
221
222 ALTER TABLE Episode
223     ADD PRIMARY KEY (content_id),
224     ALTER COLUMN show_id SET NOT NULL,
225     ALTER COLUMN season_number SET NOT NULL,
226     ALTER COLUMN episode_number SET NOT NULL,
227     ALTER COLUMN title SET NOT NULL,
228     ALTER COLUMN release_date SET NOT NULL,
229     ADD FOREIGN KEY (content_id) REFERENCES Content(content_id)
230         ON DELETE CASCADE ON UPDATE CASCADE,
231     ADD FOREIGN KEY (show_id) REFERENCES TV_Show(show_id)
232         ON DELETE CASCADE ON UPDATE CASCADE,
233     ADD FOREIGN KEY (season_number, show_id) REFERENCES Season(season_number,
234         ↪ show_id)
235         ON DELETE CASCADE ON UPDATE CASCADE;
236
237 INSERT INTO Episode VALUES
238     (6, 1, 1, 1, 'The Beginning', '2022-01-15'),
239     (7, 2, 1, 1, 'First Contact', '2021-07-10'),
240     (8, 3, 1, 1, 'Laugh Riot', '2020-03-05'),
241     (9, 4, 1, 1, 'Empire Rises', '2023-02-14'),
242     (10, 5, 1, 1, 'The Spell', '2021-11-11');
243
244 CREATE TABLE Watch_History (
245     user_id             INT,
246     content_id          INT,
247     session_start_timestamp TIMESTAMP,
248     from_t              FLOAT,
249     to_t                FLOAT,
250     IsCompleted         BOOLEAN
251 );
252
253
254
255
256
257
258
259
260

```

```

261 ALTER TABLE Watch_History
262     ADD PRIMARY KEY (user_id, content_id, session_start_timestamp),
263     ALTER COLUMN from_t SET NOT NULL,
264     ALTER COLUMN to_t SET NOT NULL,
265     ALTER COLUMN IsCompleted SET NOT NULL,
266     ADD FOREIGN KEY (user_id) REFERENCES Users(user_id)
267         ON DELETE CASCADE ON UPDATE CASCADE,
268     ADD FOREIGN KEY (content_id) REFERENCES Content(content_id)
269         ON DELETE CASCADE ON UPDATE CASCADE;
270
271 INSERT INTO Watch_History VALUES
272     (1, 1, '2025-09-02 10:30:00', 03.0, 20.0, TRUE),
273     (1, 2, '2025-09-02 10:30:00', 03.0, 20.0, FALSE),
274     (2, 3, '2025-09-02 10:30:00', 03.0, 20.0, TRUE),
275     (3, 1, '2025-09-02 10:30:00', 03.0, 20.0, TRUE);
276
277 CREATE OR REPLACE FUNCTION check_tot_duration()
278 RETURNS TRIGGER AS
279 $$
280 DECLARE
281     content_duration DECIMAL(5,2);
282 BEGIN
283     -- Fetch the duration (must exist due to FK)
284     SELECT duration INTO content_duration
285     FROM Content
286     WHERE content_id = NEW.content_id;
287
288     -- Enforce to_t duration
289     IF NEW.to_t > content_duration THEN
290         RAISE EXCEPTION
291             'Watch_History(to_t) cannot be greater than Content(duration)';
292     END IF;
293
294     RETURN NEW;
295 END;
296 $$
297 LANGUAGE plpgsql;
298
299 CREATE TRIGGER enforce_tot_duration_limit
300 BEFORE INSERT OR UPDATE ON Watch_History
301 FOR EACH ROW
302 EXECUTE FUNCTION check_tot_duration();

```

5.2 Defining the database

In this section both primary keys and foreign keys are defined. In this section, constraints and triggers define certain rules for the database. Further more reasoning for datatypes will be discussed.

5.2.1 Streaming_Platform

For the relation `Streaming_Platform`, the `ALTER TABLE` is used to firstly add a primary key to `platform_id`. A constraint is added for both `monthly_active_users` and `revenue`, to check that they will remain 0 or more. This is to enforce that streaming platform cannot have negative monthly active user or negative revenue. Furthermore, all attributes that are not the primary key, are set enforced as `NOT NULL`. The attribute `platform_id` is set as an integer, an id that can easily differs from all other `platform_id`'s. Both the `name` and the `headquarters` are set to `VARCHAR(20)` and `VARCHAR(120)`, this is where the streaming platform can store its name and where the platforms headquarters is located. The `monthly_active_users` attribute is set to an integer since no float (e.g. 0.5) amount of users can be active. Lastly the `revenue` attribute is set to a decimal, with 15 digits and 2 decimals after the decimal point, this is preferred since a streaming platform can earn income in decimals (e.g. \$0.5), but not e.g. \$0.0006.

5.2.2 Users

The relation `Users` is altered to have a primary key on `user_id`, as well as enforcing `NOT NULL` on all other attributes. Furthermore the attribute `email` is constrained as a `UNIQUE` to ensure that multiple users cannot have the same email address. A constraint for the type attribute is also made to ensure that a user which is not free, premium or creator, cannot be created. This constraint enforces that the type attribute, must either be `'Free_User'`, `'Premium_User'`, `'Creator_User'`. The attribute `user_id` is set to an integer, similar to the `platform_id`, this is to have easy distinguishability. The birthdate attribute is set to `DATE` and all set to `VARCHARS()` for information about the user.

5.2.3 Free_User

The `Free_User` relation is altered to have a primary key on `user_id`. Attributes that are not primary keys are altered to enforce `NOT NULL`. The attribute `user_id` is altered to a foreign key to `Users(user_id)`. Finally two constraints are made to enforce that a free users ad frequency is never negative and that the free users can watch a maximum resolution of 720p. Since `user_id` is a reference to `Users(user_id)` so it is also set as an int. The attribute `ad_frequency` is set to a float since it will allow a administrator to be more specific. The `max_resolution` is set as an integer and follows standard resolution specifications. For example $1080(p) = 1920 \times 1080$ or $720(p) = 1280 \times 720$,

5.2.4 Premium_User

The `Premium_User` relation is altered to have a primary key on `Users(user_id)`, and to set all other attributes to `NOT NULL`. The attribute `user_id` is altered to a foreign key to `Users(user_id)`, and the attribute `monthly_fee` is enforced to no be negative by a constraint. Since `user_id` is a reference to `Users(user_id)` so it is also set as an int. Both `subscription_plan` and `payment_method` are set to `VARCHAR()`, which is just information about that user. The `monthly_fee` attribute is set as a decimal, much like the `Streamin_Platform(revenue)`.

5.2.5 Creator_User

The `Creator_User` relation is altered to have a primary key on `user_id`, and all other attributes are altered and set to `NOT NULL`. Finally a foreign key is put on `user_id` referencing `Users(user_id)`. Since `user_id` is a reference to `Users(user_id)` so it is also set as an int. Both `uploaded_content_count` and `total_views` are set it integers as they cannot be decimals, since both a creator cannot upload 3.5 pieces of content. The `earnings` attribute is set as a decimal, much like the `Streamin_Platform(revenue)`.

5.2.6 Content

The **Content** relation is altered to have a primary key on **content_id**, and all other attributes are altered and set to **NOT NULL**. A foreign key is put on **platform_id**, referencing **Streaming_Platform(platform_id)**, as well as a foreign key on **uploaded_By**, referencing **Creator_User(user_id)**. The primary key (**content_id**) is set to int as it is easy distinguishable, much like the **user_id**. **platform_id** is a reference to **Streaming_Platform(platform_id)** so it is also set as an integer. The attribute **type** is set as a **VARCHAR(20)** (e.g. **'Movie'**) and the **duration** attribute is set to **DECIMAL(5,2)**. This is done so that a content can have a duration from 0.00 minutes to 999.99 minutes. The **IsFree** attribute is set to a boolean and is only either free or not. Lastly the **uploadedBy** is referencing the **Users(user_id)** so this will set to an integer.

5.2.7 Movie

For the **Movie** relation a primary key on **content_id**, and all other attributes are altered and set to **NOT NULL**. A foreign key is put on **content_id** referencing **Content(content_id)** and constraint enforcing that rating is neither negative nor more than 10.0. The **content_id** is set as an integer because it is a reference to the **Content(content_id)**. The **release_year** is set to an integer. The 3 attributes **title**, **genre** and **director** are set to the type **VARCHAR()** for information about a movie. Lastly the **rating** attribute is set to **DECIMAL(3,1)** storing the rating information at only 2 digits and 1 decimal, meaning that without the constraint you could have a rating ranging from 0.0 to 99.9.

5.2.8 TV_Show

The **TV_Show** relation is altered to have a primary key on **show_id** and all other attributes are set to **NOT NULL**. This relation doesn't need anymore since it is very simple. Besides the private key **show_id** which is an integer all attributes mainly just hold information about the TV show, these are set to the type **VARCHAR()**

5.2.9 Season

The **Season** relation is altered to have a composite primary key on **season_number** and **show_id**, this is necessary to point what season in which TV-show. Further more a foreign key is placed on **show_id** referencing the **TV_Show(show_id)**. The **season_number** is used to identify the specific season so it is like many other id's set as an integer, and because **show_id** is a reference to **TV_Show(show_id)** this will also be an integer. The final attribute **release_date** is much alike the **Movie(release_year)**, only this is a specific date, so it is set to the type **DATE**.

5.2.10 Episode

The **Episode** relation is altered to have a primary key on the attribute **content_id**. All other attributes are set to **NOT NULL** as 3 foreign keys. The attribute **content_id** is altered to a foreign key referencing the **Content(content_id)**. **show_id** is also altered to have a foreign key referencing **TV_Show(show_id)**. The last foreign key is placed on both **season_number** and **show_id**. These two foreign keys are both referencing **Season(season_number)** and **TV_Show(show_id)**.

5.2.11 Watch_History

The relation **Watch_History** contains information about a timestamp of a user watching a specific piece on content. The relation is altered with a composite primary key on **user_id**, **content_id** and **session_start_timestamp**. This is so that one watch history is distinguishable from another. Besides the primary keys, all other attributes are altered and set to **NOT NULL**. The attribute **user_id** is altered to have a foreign key referencing **Users(user_id)**, and **content_id** is altered to have a foreign key referencing **Content(content_id)**. Therefore both the **user_id** and **content_id** are set to integers. The primary key **session_start_timestamp** is set to the type **TIMESTAMP**. The **TIMESTAMP** datatype is great in this situation, because it captures both date and time in one type. The attributes **from_t** and **to_t** are much like **Content(duration)**, set to the type **DECIMAL(5,2)**. This will make it so that these attributes are easily comparable. Finally the attribute **IsCompleted** is set to the type **BOOLEAN**.

5.2.12 Trigger enforce_tot_duration_limit

The trigger `enforce_tot_duration_limit` is created to ensure that the attribute `to_t` in the `Watch_History` relation does not exceed the duration of the associated content. The trigger is defined to execute **before** either an `INSERT` or an `UPDATE` operation on the `Watch_History` table. This is done on a per-row basis using the clause `FOR EACH ROW`. When activated, the trigger executes the function `check_tot_duration()`.

The function `check_tot_duration()` is written in PLPGSQL and declared to return a `TRIGGER`. Inside the function, a local variable `content_duration` of type `DECIMAL(5,2)` is declared. This variable is used to store the duration of the content being referenced in the new row. The `SELECT INTO` statement is used to retrieve the `duration` attribute from the `Content` relation, where the `content_id` matches the one in the new tuple being inserted or updated. A conditional `IF` statement then checks if the `to_t` value in the new watch history record exceeds the actual duration of the content. If this condition is true, the function raises an exception with a descriptive error message: *Watch_History(to_t) cannot be greater than Content(duration)*. This enforces data integrity by ensuring that a user cannot record a watch session that claims to have watched beyond the end of the content.

Finally, the function returns the new tuple using `RETURN NEW;`, which allows the insert or update to proceed if no constraint was violated.

6 Part 3: Querying

This part executes the following SQL queries. To help with the readability and structure of your SQL queries you can use WITH statements. Other features of PostgreSQL that you should consider using is CASE and EXTRACT date_part. You should document in the report the SQL queries that you used in order to answer the corresponding questions and also include a small description of what you are trying to achieve with each query. You should also include screenshots of the query results.

6.0.1 i - Movies and TV-Shows

Retrieve a list with all the Movies and TV-Shows Offered by Streaming Platforms. The result should consist of the Platform Name, the Title of the Movie or TV Show and the type of content.

Answer:

```
1 SELECT *
2 FROM (
3     -- Movies
4     SELECT
5         sp.name      AS platform_name,
6         m.title      AS content_title,
7         'Movie'      AS content_type
8     FROM Content c
9     JOIN Streaming_Platform sp ON c.platform_id = sp.platform_id
10    JOIN Movie m              ON c.content_id  = m.content_id
11
12    UNION ALL
13
14    -- TV Shows (one row per show/platform)
15    SELECT DISTINCT
16        sp.name      AS platform_name,
17        ts.title     AS content_title,
18        'TV Show'    AS content_type
19    FROM Content c
20    JOIN Streaming_Platform sp ON c.platform_id = sp.platform_id
21    JOIN Episode e           ON c.content_id  = e.content_id
22    JOIN TV_Show ts          ON e.show_id     = ts.show_id
23 ) AS offerings
24 ORDER BY
25     platform_name,
26     content_title;
```

The SQL query above constructs a unified, ordered listing of every movie and tv-show available on each streaming platform. It begins by defining a table named `offerings`, which consists of two component queries combined with `UNION ALL`. The first component retrieves all movies, by selecting the platform name from `Streaming_Platform`, the movie title from `Movie`, and string “Movie” as the content type. These tables are joined through the `Content` table, which provides the association between content items and their platforms. The join sequence ensures that only items classified as movies appear in this part of the result set, and it allows the query to display the platform name alongside each movie title.

Next, the query retrieves television shows in much the same way: it joins `Content` with `Streaming_Platform` to get the platform name, then joins `Episode` to select only episode entries, and finally joins `TV_Show` to obtain each show’s title. Since a single show can consist of many episodes, `SELECT DISTINCT` is used so that each show appears only once per platform, and the literal “TV Show” is assigned as the content type to match the movie labeling.

The choice of `UNION ALL` rather than plain `UNION` allows the query to concatenate the two component result sets without performing an extra de-duplication step; since the movie and TV-show subsets are inherently disjoint, no unwanted duplicates exist and performance is improved.

Making these two queries within a single table permits the application of one global `ORDER BY` clause. After the union, the outer `SELECT *` retrieves all columns from `offerings` and orders the combined rows first by platform name and then by content title. This final sort operation produces a neat listing in which each platform’s offerings are grouped together and each title appears in alphabetical order.

6.0.2 ii - Counting uploads

Assume that the attribute `uploaded_content_count` of relation `Creator_User` was not available. Write a SQL query that calculates for each user how many content files she/he has uploaded. The query should return the User ID, the Users first and last name and the number of uploaded files

Answer:

```
SELECT
    u.user_id,
    u.first_name,
    u.last_name,
    COUNT(c.content_id) AS uploaded_files_count
FROM
    Users u
LEFT JOIN
    Content c
ON u.user_id = c.uploadedBy
GROUP BY
    u.user_id,
    u.first_name,
    u.last_name
ORDER BY
    uploaded_files_count DESC;
```

The query above produces a ranked list of every user alongside the total number of content files they have uploaded. It begins by selecting each user's identifier and their first and last names from the `Users` table. A `LEFT JOIN` to the `Content` table on the condition that the user's ID matches the `uploadedBy` field ensures that even users who haven't uploaded anything still appear in the output. For each user, `COUNT(c.content_id)` counts how many content rows reference them as the uploader, and this count is aliased as `uploaded_files_count`. Grouping by the user's ID and name fields aggregates all of each user's join matches into a single row, and finally the result is sorted in descending order of `uploaded_files_count` so that the most prolific uploaders appear at the top.

6.0.3 iii - Complete views

Retrieve for each content id, the number of complete views, where the number of complete views is less than 2. The query should return the Content ID, and the number of complete views.

Answer:

```
1  SELECT
2      c.content_id,
3      COALESCE(w.complete_views, 0) AS complete_views
4  FROM
5      Content c
6  LEFT JOIN (
7      SELECT
8          content_id,
9          COUNT(*) AS complete_views
10     FROM
11         Watch_History
12     WHERE
13         IsCompleted = TRUE
14     GROUP BY
15         content_id
16     HAVING
17         COUNT(*) < 2
18 ) w
19     ON c.content_id = w.content_id
20 ORDER BY
21     complete_views, c.content_id;
```

This query returns every content item together with the number of times it has been fully watched—showing zero for items with no or only incomplete views—and filters that count to under two. It starts by aggregating completed views in a derived table: for each `content_id` in `Watch_History` where `IsCompleted = TRUE`, it counts how many such records exist and keeps only those with fewer than two complete views. The outer query then performs a `LEFT JOIN` from the full `Content` list to this derived table, matching on `content_id`. By using `COALESCE(w.complete_views, 0)`, any content without a matching entry in the derived table is given a zero count. Finally, the result is ordered first by the number of complete views (ascending) and then by `content_id` to provide a concise, sorted overview of all content complete-view-count.

6.0.4 iv - Max views on date

Write a SQL query that finds the date/dates of 2024 with the maximum number of content views. This query should return the date and the number of views. If two dates have a tie, both of them should appear in the result.

```
1      SELECT
2      view_date,
3      views
4  FROM (
5      SELECT
6          CAST(session_start_timestamp AS DATE)      AS view_date,
7          COUNT(*)                                  AS views,
8          RANK() OVER (ORDER BY COUNT(*) DESC)        AS rnk
9      FROM
10         Watch_History
11     WHERE
12         session_start_timestamp BETWEEN '2024-01-01' AND '2024-12-31'
13     GROUP BY
14         CAST(session_start_timestamp AS DATE)
15 ) sub
16 WHERE
17     rnk = 1;
18
```

The query above identifies the date or dates in 2024 on which the highest number of watch sessions occurred. The inner subquery begins by converting each session's timestamp into a calendar date and counting how many sessions fell on that date. It then applies the window function `RANK() OVER (ORDER BY COUNT(*) DESC)` to assign a rank of 1 to the day(s) with the greatest count, a rank of 2 to the next highest, and so on. The `WHERE` clause limits the data to timestamps between January 1 and December 31, 2024, and the `GROUP BY` clause aggregates sessions by date. In the outer query, we simply select `view_date` and `views` from this ranked result where `rnk = 1`, ensuring that every date tied for the maximum number of views in 2024 is returned.

6.0.5 v - UserRecord

Create a View for the UserRecord of 1.2. To simplify this query let us consider that the number_of_views corresponds to how many times a user has completely watched this content.

Answer:

```
1 CREATE VIEW UserRecord AS
2 WITH
3     -- Count of completed views per user/content
4     view_counts AS (
5         SELECT
6             user_id,
7             content_id,
8             COUNT(*) AS number_of_views
9         FROM
10            Watch_History
11        WHERE
12            IsCompleted = TRUE
13        GROUP BY
14            user_id,
15            content_id
16    ),
17
18    -- All user-content pairs, either by watching or uploading
19    user_content AS (
20        SELECT DISTINCT
21            user_id,
22            content_id
23        FROM
24            Watch_History
25        WHERE
26            IsCompleted = TRUE
27
28        UNION
29
30        SELECT
31            uploadedBy AS user_id,
32            content_id
33        FROM
34            Content
35    )
36
37 SELECT
38     u.user_id,
39     u.first_name,
40     u.last_name,
41     uc.content_id,
42
43     CASE
44         WHEN c.uploadedBy = u.user_id THEN TRUE
45         ELSE FALSE
46     END AS IsUploaded,
47
48
49
50
```

```

51  CASE
52      WHEN m.title IS NOT NULL THEN m.title
53      ELSE e.title
54  END AS title,
55  e.show_id,
56  e.season_number,
57  e.episode_number,
58
59  CASE
60      WHEN vc.number_of_views IS NOT NULL THEN vc.number_of_views
61      ELSE 0
62  END AS number_of_views
63
64  FROM
65      user_content uc
66
67  JOIN Users u
68      ON uc.user_id = u.user_id
69
70  JOIN Content c
71      ON uc.content_id = c.content_id
72
73  LEFT JOIN view_counts vc
74      ON vc.user_id = uc.user_id AND vc.content_id = uc.content_id
75
76  LEFT JOIN Movie m
77      ON m.content_id = c.content_id
78
79  LEFT JOIN Episode e
80      ON e.content_id = c.content_id;
81

```

The **CREATE VIEW** `UserRecord` statement defines a database view with all user's profile information, the content items they either have uploaded or watched, and the number of times they completed those items. It begins by declaring two common table expressions (CTE).

The first CTE, `view_counts`, filters through `Watch_History`, to rows where `IsCompleted` = **TRUE**, and groups by `user_id` and `content_id` to compute `number_of_views` for each user-content pair. The second CTE, `user_content`, assembles every distinct combination of `user_id` and `content_id` that arises either from a completed watch session (`Watch_History`) or from uploading the content (`Content.uploadedBy`), using a **UNION** to merge those sources without duplication.

The main **SELECT** then iterates over each (`user_id`, `content_id`) pair in `user_content`. It joins to `Users` to pull in the user's `first_name` and `last_name`, and to `Content` to connect each pair to its content record. A **LEFT JOIN** to `view_counts` brings in the precomputed `number_of_views` but leaves the field null if the user never completed that content; a **CASE** expression replaces any null with zero. Two further **LEFT JOINS** to `Movie` and `Episode` determine whether the content item is a movie (retrieving its title) or an episode (retrieving title, `show_id`, `season_number`, and `episode_number`), and a **CASE** expression chooses the appropriate title column. Finally, a **CASE** on `Content.uploadedBy` flags whether the current user is the uploader of that content, populating the `IsUploaded` boolean. When queried, `UserRecord` delivers one row per user-content pairing, complete with user details, content metadata, upload status, and the count of completed views.

7 Part 4 - Triggers

Let us assume that the UserRecord relation from Part 1.2 was part of the database. Implement using triggers, the automatic update of this relation, so that at any time it is up to date. Moreover, update automatically using triggers, the attributes `uploaded_content_count` and `total_views` of relation `Creator_User`, so that at any time they reflect reality. You should present the trigger scripts and some scenarios (including screenshots) showcasing that these triggers work. Argue about the triggers that you chose to implement and justify the decisions that you made.

Answer:

Note: The triggers in this part are separated to improve clarity and make future modifications easier. This structure helps keep the logic organized and easier to manage. Additionally, it is assumed that a new content creator has not uploaded any content at the time of creation, so their upload count is initialized to 0. Upload tracking is handled separately.

7.1 Updating UserRecord

7.2 Insertion

```
1 CREATE OR REPLACE FUNCTION trigger_update_userrecord_on_watch()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Only update on completed views
5     IF NEW.IsCompleted THEN
6         INSERT INTO UserRecord (
7             user_id, first_name, last_name, content_id,
8             IsUploaded, title, show_id, season_number, episode_number, number_of_views
9         )
10        SELECT
11            u.user_id,
12            u.first_name,
13            u.last_name,
14            c.content_id,
15            (c.uploadedBy = u.user_id) AS IsUploaded,
16            COALESCE(m.title, ts.title),
17            e.show_id,
18            e.season_number,
19            e.episode_number,
20            1
21        FROM Users u
22        JOIN Content c      ON c.content_id = NEW.content_id
23        LEFT JOIN Movie m   ON m.content_id = c.content_id
24        LEFT JOIN Episode e ON e.content_id = c.content_id
25        LEFT JOIN TV_Show ts ON e.show_id = ts.show_id
26        WHERE u.user_id = NEW.user_id
27        ON CONFLICT (user_id, content_id)
28            DO UPDATE SET number_of_views = UserRecord.number_of_views + 1;
29    END IF;
30
31    RETURN NEW;
32 END;
33 $$ LANGUAGE plpgsql;
```

```

37 CREATE TRIGGER trigger_watchhistory_insert
38 AFTER INSERT ON Watch_History
39 FOR EACH ROW
40 EXECUTE FUNCTION trigger_update_userrecord_on_watch();

```

The trigger above maintains the `UserRecord` relation by automatically updating it in response to any new completed views recorded in the `Watch_History` table. It ensures that the number of complete views for each user-content pair remains consistent with the viewing activity.

The function is invoked after each `INSERT` on the `Watch_History` relation. It begins by checking whether the inserted row has `IsCompleted` set to true. Only in such cases should the trigger proceed, thereby ensuring that only complete views are counted in the `UserRecord(number_of_views)` attribute.

Upon confirmation of a complete view, the function constructs a new entry for the `UserRecord` table. It collects the necessary information by joining `Users` to `Content` via `Watch_History(content_id)`, and then optionally joins to `Movie`, `Episode`, and `TV_Show` to retrieve relevant data. The attribute `UserRecord(IsUploaded)` is derived by checking whether the viewer is the same as the content's uploader (if `Users(user_id) = Content(uploadedBy)`). The content title is retrieved either from `Movie(title)` or `TV_Show(title)`, using `COALESCE` to accommodate both content types, since one of them are `NULL`.

The insertion is performed using an `ON CONFLICT` clause on the composite key (`user_id`, `content_id`). If a matching row already exists in `UserRecord`, the query instead increments the existing `number_of_views` value by 1, ensuring accurate accumulation of view counts without duplication.

The trigger itself is declared to activate `AFTER INSERT` on the `Watch_History` relation and is configured to execute the function once per affected row.

7.3 Deletion

```
1 CREATE OR REPLACE FUNCTION trigger_cleanup_after_watch_delete()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     uploader_id INT;
5     existing_views INT;
6 BEGIN
7     -- Only update completed views
8     IF OLD.IsCompleted THEN
9         -- Update total_views for the Creator_User
10        SELECT uploadedBy INTO uploader_id
11        FROM Content
12        WHERE content_id = OLD.content_id;
13
14        UPDATE Creator_User
15        SET total_views = total_views - 1
16        WHERE user_id = uploader_id;
17
18        -- Save existing view count in UserRecord
19        SELECT number_of_views INTO existing_views
20        FROM UserRecord
21        WHERE user_id = OLD.user_id AND content_id = OLD.content_id;
22
23        -- update UserRecord
24        IF existing_views = 1 THEN
25            DELETE FROM UserRecord
26            WHERE user_id = OLD.user_id AND content_id = OLD.content_id;
27        ELSE
28            UPDATE UserRecord
29            SET number_of_views = number_of_views - 1
30            WHERE user_id = OLD.user_id AND content_id = OLD.content_id;
31        END IF;
32    END IF;
33
34    RETURN NULL;
35 END;
36 $$ LANGUAGE plpgsql;
37
38 CREATE TRIGGER trigger_watchhistory_delete_cleanup
39 AFTER DELETE ON Watch_History
40 FOR EACH ROW
41 EXECUTE FUNCTION trigger_cleanup_after_watch_delete();
```


This trigger takes care of cleaning up when someone deletes a row from `Watch_History`, but only if that row represents a completed view—that is, when `IsCompleted` was set to `TRUE`.

When that happens, the function first looks up who uploaded the content by querying the `Content` table using the `content_id` from the deleted row. Once it finds the uploader's `user_id`, it subtracts one from their `total_views` in the `Creator_User` table. That way, the creator's total view count stays accurate.

Then it checks the `UserRecord` table to see how many times this user has viewed that same content. If the `number_of_views` is more than 1, it just decreases it by 1. But if it's exactly 1—meaning the deleted row was the only completed view—then it deletes the corresponding row from `UserRecord` entirely.

This whole setup ensures that deleting a viewing session also updates any stats or records that depend on it. It keeps the creator's view count and the user's watch history in sync with what's actually in the database.

7.4 Updating Creator_User(total_views)

```
1 CREATE OR REPLACE FUNCTION trigger_update_total_views()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     uploader_id INT;
5 BEGIN
6     IF NEW.IsCompleted THEN
7         SELECT uploadedBy INTO uploader_id
8         FROM Content
9         WHERE content_id = NEW.content_id;
10
11         UPDATE Creator_User
12         SET total_views = total_views + 1
13         WHERE user_id = uploader_id;
14     END IF;
15
16     RETURN NULL;
17 END;
18 $$ LANGUAGE plpgsql;
19
20 CREATE TRIGGER trigger_watchhistory_total_views
21 AFTER INSERT ON Watch_History
22 FOR EACH ROW
23 EXECUTE FUNCTION trigger_update_total_views();
```

This trigger mechanism ensures that the `Creator_User(total_views)` attribute accurately reflects the total number of completed views received across all content uploaded by each user. It achieves this by reacting to insertions into the `Watch_History` relation, updating the view count only when the inserted session indicates a completed view.

The function `trigger_update_total_views` is executed **AFTER INSERT** on the `Watch_History` table. It begins by checking whether the newly inserted row has the `IsCompleted` attribute set to true, thereby filtering the trigger's effect to completed views only. This design ensures that only meaningful user engagements—i.e., full views—contribute to the popularity statistics of uploaded content.

If this condition is satisfied, the function queries the `Content` relation to retrieve the `uploadedBy` value corresponding to the newly viewed `Watch_History(content_id)`. The result is stored in the local variable `uploader_id`, representing the user who originally uploaded the content in question.

Subsequently, the function updates the `Creator_User(total_views)` field by incrementing it by one, but only for the user whose `Creator_User(user_id)` matches the identified `uploader_id`. This update operation maintains a running total of how many completed views the creator has accumulated across all their contributions.

The trigger itself is created with the name `trigger_watchhistory_total_views` and is set to execute the function `trg_update_total_views` for each row inserted into the `Watch_History` table. This row-level granularity ensures that the total views metric is immediately and automatically updated with each new completed viewing session, preserving the consistency and accuracy of creator analytics.

7.5 Updating Creator_User(uploaded_content_count)

```
1 CREATE OR REPLACE FUNCTION trigger_update_uploaded_count()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF OLD.uploadedBy IS DISTINCT FROM NEW.uploadedBy THEN
5         -- Add count for the old uploader
6         UPDATE Creator_User
7         SET uploaded_content_count = uploaded_content_count - 1
8         WHERE user_id = OLD.uploadedBy;
9
10        -- Subtract count for the new uploader
11        UPDATE Creator_User
12        SET uploaded_content_count = uploaded_content_count + 1
13        WHERE user_id = NEW.uploadedBy;
14    END IF;
15
16    RETURN NULL;
17 END;
18 $$ LANGUAGE plpgsql;
19
20 CREATE TRIGGER trigger_content_upload_count
21 AFTER UPDATE ON Content
22 FOR EACH ROW
23 WHEN (OLD.uploadedBy IS DISTINCT FROM NEW.uploadedBy)
24 EXECUTE FUNCTION trigger_update_uploaded_count();
```

This trigger ensures that the `Creator_User(uploaded_content_count)` attribute remains accurate when the uploader of a content item is modified in the `Content` table. It is implemented through a PL/pgSQL function and an associated trigger that respond specifically to `UPDATE` operations on the `Content` relation.

The function is executed after an update and first verifies whether the `Content(uploadBy)` field has changed, by comparing the previous value (`OLD.uploadedBy`) to the new value (`NEW.uploadedBy`). If a change in uploader is detected, the function performs two operations: it decrements the `Creator_User(uploaded_content_count)` for the user identified by the old uploader value, and it increments the same attribute for the user corresponding to the new uploader value. This dual adjustment ensures that each user's content upload count accurately reflects the number of content items currently associated with them.

The trigger is defined on the `Content` table and is fired `AFTER UPDATE`. It is configured with a `WHEN` clause that restricts execution to updates where the `uploadBy` field has changed. Specifically, it uses the condition `WHEN (OLD.uploadedBy IS DISTINCT FROM NEW.uploadedBy)` to detect meaningful changes. This avoids unnecessary trigger executions when other, unrelated fields in the `Content` table are updated.

Together, the trigger and function maintain consistency between the `Content` and `Creator_User` relations, ensuring that `uploaded_content_count` is automatically updated to reflect the current state of the database.

7.6 Updating Content

```
1 CREATE OR REPLACE FUNCTION trigger_increment_uploaded_count()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     UPDATE Creator_User
5     SET uploaded_content_count = uploaded_content_count + 1
6     WHERE user_id = NEW.uploadedBy;
7
8     RETURN NULL;
9 END;
10 $$ LANGUAGE plpgsql;
11
12 CREATE TRIGGER trigger_content_insert_count
13 AFTER INSERT ON Content
14 FOR EACH ROW
15 EXECUTE FUNCTION trigger_increment_uploaded_count();
```

This trigger makes sure we keep track of how much stuff a creator has uploaded. Right now, we already have a trigger that handles when a content item's uploader changes (like when someone reassigns a video to a different creator), but we didn't yet handle what happens when a brand-new row is inserted into `Content`.

That's what this trigger does. When new content is added, the trigger grabs the `uploadedBy` user ID from the new row and increments the `uploaded_content_count` for that creator in the `Creator_User` table by 1.

This way, every time a creator uploads something new, their content count automatically increases without anyone needing to manually update it. Simple and clean—just how it should be.

8 Part 5 - Index

For the following SQL queries, investigate if you can speed up their execution by building some index. If so, implement the index. In any case, argue about your choice and justify your decisions. You should include the CREATE INDEX scripts.

8.1 Defining indexing

An SQL index is used in databases to improve the speed of data retrieval. It works similar to finding something in a book. The book might be split up into chapters, making it easy to find a specific part of the book. This is done by sorting the data. This helps the database find specific information faster without having to check every row in a table. An index is created on one or more columns in a table. When a query searches for data in those columns, the database can (with the index) locate the results very quickly.

Indexes are mostly useful when a column is frequently used in search conditions, sorting, or grouping. They are also helpful for improving performance when the table contains a large amount of data. However, indexes also take up additional storage space and can therefore slow down data changes such as inserting, updating, or deleting rows, because the index must also be updated. In summary, indexes improve the speed of reading data but can make writing data slightly slower. Because of this, they should be used when they provide a clear benefit to query performance (speed).

When a primary key is created, the database automatically creates a unique index on that column. This means that values are unique and therefore it is not necessary to create a separate index on a primary key, since it's already indexed.

In short, indexes *should* be created on foreign keys or clauses like **WHERE**, **JOIN** or **GROUP BY**

8.1.1 ii - Counting uploads

Assume that the attribute `uploaded_content_count` of relation `Creator_User` was not available. Write a SQL query that calculates for each user how many content files she/he has uploaded. The query should return the User ID, the Users first and last name and the number of uploaded files

Answer:

```
1  SELECT
2      u.user_id,
3      u.first_name,
4      u.last_name,
5      COUNT(c.content_id) AS uploaded_files_count
6  FROM
7      Users u
8  LEFT JOIN
9      Content c
10     ON u.user_id = c.uploadedBy
11 GROUP BY
12     u.user_id,
13     u.first_name,
14     u.last_name
15 ORDER BY
16     uploaded_files_count DESC;
```

The query above was made in part 2: ii

```
1  CREATE INDEX idx_content_uploadedBy
2      ON Content(uploadBy);
```

This index above is placed on the `Content(uploadBy)` column in the `Content` table because of its `JOIN` clause.

8.1.2 iii - Complete views

Retrieve for each content id, the number of complete views, where the number of complete views is less than 2. The query should return the Content ID, and the number of complete views.

Answer:

```
1 SELECT
2     c.content_id,
3     COALESCE(w.complete_views, 0) AS complete_views
4 FROM
5     Content c
6 LEFT JOIN (
7     SELECT
8         content_id,
9         COUNT(*) AS complete_views
10    FROM
11        Watch_History
12   WHERE
13       IsCompleted = TRUE
14   GROUP BY
15       content_id
16   HAVING
17       COUNT(*) < 2
18 ) w
19 ON c.content_id = w.content_id
20 ORDER BY
21     complete_views, c.content_id;
```

The query above was made in part 2: iii

```
1 CREATE INDEX idx_isCompleted
2     ON WatchHistory(IsCompleted, content_id)
```

Indexing WatchHistory(IsCompleted) and WatchHistory(content_id) helps because the query filters by WatchHistory(IsCompleted = TRUE) and groups by content_id. The index speeds up both the filtering and grouping, making the query faster.

8.1.3 iv - Max views on date

Write a SQL query that finds the date/dates of 2024 with the maximum number of content views. This query should return the date and the number of views. If two dates have a tie, both of them should appear in the result.

Answer:

```
1 SELECT
2     view_date,
3     views
4 FROM (
5     SELECT
6         CAST(session_start_timestamp AS DATE)      AS view_date,
7         COUNT(*)                                    AS views,
8         RANK() OVER (ORDER BY COUNT(*) DESC)       AS rnk
9     FROM
10        Watch_History
11     WHERE
12        session_start_timestamp BETWEEN '2024-01-01' AND '2024-12-31'
13     GROUP BY
14        CAST(session_start_timestamp AS DATE)
15 ) sub
16 WHERE
17     rnk = 1;
```

The query above was made in part 2: iv.

```
1 CREATE INDEX idx_watch_history_session_date
2     ON Watch_History(session_start_timestamp);
```

The index above is placed on on the WatchHistory(session_start_timestamp) column in the Watch_History table. It helps the database quickly filter and group data by session start time, which improves performance for queries using date ranges or sorting by timestamp.